OS Orange Report

Name: Srikanth Balaaji

SRN: PES2UG24CS518

Section: I

Screenshot 1A:

```
vboxuser@Ubuntu:~/Desktop/OS_Orange/PES2UG24CS518-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check
```

Screenshot 1B:

```
vboxuser@Ubuntu:~/Desktop/OS_Orange/PES2UG24CS518-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

Screenshot 2A:

```
vboxuser@Ubuntu:~/Desktop/OS_Orange/PES2UG24CS518-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B:

```
vboxuser@Ubuntu:~/Desktop/OS_Orange/PES2UG24CS518-pes-vcs$ xxd .pes/objects/d5/8
213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
```

Screenshot 3A:

```
vboxuser@Ubuntu:~/Desktop/OS_Orange/PES2UG24CS518-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  object.c
  untracked:  tree.c
  untracked:  test_sequence.sh
  untracked:  pes.c
  untracked:  Makefile
  untracked:  commit.c
  untracked:  tree.h
  untracked:  .gitignore
  untracked:  README.md
  untracked:  hello.txt
  untracked:  file.txt
  untracked:  test_tree.c
  untracked:  test_objects.c
```

Screenshot 3B:

```
vboxuser@Ubuntu:~/Desktop/OS_Orange/PES2UG24CS518-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 17767340
18 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 17767340
28 6 file2.txt
```

Screenshot 4A:

```
vboxuser@Ubuntu:~/Desktop/OS_Orange/PES2UG24CS518-pes-vcs$ ./pes log
commit ba767d33948b81739fec46d8ca301a1160dc9b656263a287026946c45ac2a768
Author: PES User <pes@localhost>
Date:   1776735753

    Add farewell

commit 6339cea59545fbc56ff8247eb1599afdaa3946e37a841cd3793491b61d0eada2
Author: PES User <pes@localhost>
Date:   1776735720

    Add worldell

commit 046549a67dbc4c4646052fb3b5ff0dae3895527f7052d19a7c4e142ca84ecc0e
Author: PES User <pes@localhost>
Date:   1776735702
```

Screenshot 4B:

```
vboxuser@Ubuntu:~/Desktop/OS_Orange/PES2UG24CS518-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/04/6549a67dbc4c4646052fb3b5ff0dae3895527f7052d19a7c4e142ca84ecc0e
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/50/da219c4c66a82940f7a8555908504a9bb1ede3773efe8669ad72f1d54966b3
.pes/objects/63/39cea59545fbc56ff8247eb1599afdaa3946e37a841cd3793491b61d0eada2
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7c/83d6df2d741034233b181c4bfa7a6e64a49ec875cc62a850886d11af72429c
.pes/objects/ba/767d33948b81739fec46d8ca301a1160dc9b656263a287026946c45ac2a768
.pes/objects/c1/5daab6ccf47288c923fa5384e07d0e7c63a4f443a1a77c77d51e67b257e822
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

Screenshot 4C:

```
vboxuser@Ubuntu:~/Desktop/OS_Orange/PES2UG24CS518-pes-vcs$ cat .pes/refs/heads/main
ba767d33948b81739fec46d8ca301a1160dc9b656263a287026946c45ac2a768
vboxuser@Ubuntu:~/Desktop/OS_Orange/PES2UG24CS518-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Screenshot of make test-integration command:

```
vboxuser@Ubuntu:~/Desktop/OS_Orange/PES2UG24CS518-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 9661de5500ce... Initial commit

Log after first commit:
commit 9661de5500ceb2f130cc82e869f5468e09abcb56accf2e2ceb0e760e0a51146b
Author: PES User <pes@localhost>
Date:   1776736222
```

```
    Initial commit


--- Second Commit ---
Committed: b40fae6666c7... Update file.txt

--- Third Commit ---
Committed: a35f5e44d965... Add farewell

--- Full History ---
commit a35f5e44d965a1ad897cc6936d512064519cd2c293b6c83866314b245029c3d1
Author: PES User <pes@localhost>
Date:   1776736222

    Add farewell

commit b40fae6666c7b969b179ca7749af424c2adfddd70ddb4f4b80c538836f464ef7
Author: PES User <pes@localhost>
Date:   1776736222

    Update file.txt

commit 9661de5500ceb2f130cc82e869f5468e09abcb56accf2e2ceb0e760e0a51146b
Author: PES User <pes@localhost>
Date:   1776736222

    Initial committial commiQ
```

```
--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
a35f5e44d965a1ad897cc6936d512064519cd2c293b6c83866314b245029c3d1

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/84/c5fed72c2dd9a3130e2e9b98033827f62793fb80d0295ccedf5fc7972d96eb
.pes/objects/96/61de5500ceb2f130cc82e869f5468e09abcb56accf2e2ceb0e760e0a51146b
.pes/objects/a3/5f5e44d965a1ad897cc6936d512064519cd2c293b6c83866314b245029c3d1
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/b4/0fae6666c7b969b179ca7749af424c2adfddd70ddb4f4b80c538836f464ef7
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/de/34fb483a5c16ce29ec3195dd505dfda7e35cc5fe16a99da4b2455b332a93cf
.pes/objects/ec/0af37b159ae92a5850ceaa6e497860729d6158aab2c6833b6068a1e3c5e8ef

=== All integration tests completed ===
```

Analysis questions:

A5.1)

Two files change in .pes/: HEAD gets updated to ref: refs/heads/<branch>, and if the branch is new, a file is created at .pes/refs/heads/<branch> containing the target commit hash. The working directory must then be updated by reading the target commit's tree object, comparing it with the current tree, and writing/deleting files to match. The complexity comes from three things: handling files that exist in one branch but not the other (create or delete), handling modified files that differ between branches (conflict detection), and doing all of this atomically so a failure midway doesn't leave the repo in a broken state.

A5.2)

For each file in the current index, compute its current on-disk hash and compare it to the staged hash in the index — if they differ, the file is dirty. Then check if that same file's hash in the target branch's tree differs from the current branch's tree. If both are true (file is locally modified AND differs between branches), checkout must refuse. This requires no extra state — only the index for current staged hashes, stat() for fast metadata comparison, and object_read() on both trees for the branch comparison.

A5.3)

In detached HEAD state, HEAD contains a raw commit hash instead of ref: refs/heads/<branch>. New commits are written correctly and each points to its parent, but no branch file is updated to track them. Once you switch branches, these commits become unreachable — no ref points to them, so they're invisible to pes log and eligible for garbage collection. Recovery is possible if you remember the commit hash (it was printed at commit time) — you create a new branch pointing to it: git branch recovery <hash>, then check it out.

A6.1)

This is a mark-and-sweep approach:

1. Mark phase: Start from all branch refs in .pes/refs/ and HEAD. For each commit, add its hash to a reachable set, then recurse into its tree — adding all tree and blob hashes — then follow the parent pointer and repeat.
2. Sweep phase: List every object file in .pes/objects/, and delete any whose hash is not in the reachable set.

A hash set (like a C unordered_set or a bitmap over truncated hashes) is ideal for O(1) lookup during marking. For 100,000 commits with 50 branches: assuming ~10 files per commit, you'd visit roughly 100,000 commits + 100,000 trees + 1,000,000 blobs ≈ ~1.2 million objects in the worst case.

A6.2)

Consider this sequence:

1. A commit operation calls object_write to write a new **blob** object to .pes/objects/ — the file now exists on disk.
2. GC runs its mark phase — it starts from all branch refs. The blob isn't reachable yet because the commit object referencing it hasn't been written yet.
3. GC's sweep phase deletes the blob as unreachable.
4. The commit operation finishes writing the commit object, which now references a deleted blob — the repository is corrupt.

Git avoids this by using a grace period: GC never deletes objects newer than 2 weeks old (by checking file mtime), giving in-progress operations time to complete. It also uses .git/gc.pid lockfiles to prevent concurrent GC runs.